DOCUMENTACIÓN TÉCNICA

Manual del Programador FAFI 1.6

Team T Studio · versión 2.0.0 · noviembre de 2026

Guía técnica para entender, ejecutar, modificar y desplegar el proyecto. Para jugar, ver el Manual de Usuario.

Índice

1. Resumen y tecnologías
2. Estructura del repositorio y convención de nombres
3. Cómo ejecutarlo
4. Arquitectura del cliente
5. Modos de juego
6. Armas, personajes y skins
7. Sistema de cajas
8. Modo historia y escena WW2
9. Sonido
10. Backend y base de datos
11. Sincronización del juego con la base de datos
12. Docker y despliegue
13. Guías: cómo agregar contenido
14. Herramientas
15. Pruebas y depuración
16. Límites conocidos y pendientes

---

1. Resumen y tecnologías

FAFI 1.6 es un FPS táctico que corre **en el navegador**, sin plugins ni instalación.

Capa	Tecnología
Juego (cliente)	HTML5, CSS, **JavaScript con módulos ES** y **Three.js r160** (WebGL)
Sonido	API **Web Audio** (muestras reales + reverb sintética)
Modelos	**glTF/GLB** (mapas, personajes, armas, caja, cinemáticas)
Servidor	**PHP 8** (API REST + WebSocket Ratchet), **Nginx**, **MySQL 8**, **Redis**
Despliegue	**Docker Compose**
Sitio del equipo	HTML/CSS/JS estático (`dev-website`)

El cliente funciona **solo** (modo sin conexión, progreso en `localStorage`) o **con servidor** (cuentas, inventario, cajas y ranking en MySQL).

---

2. Estructura del repositorio y convención de nombres

```
Fafi-1.6/
├── game-client/           El juego (estático)
│   ├── index.html         Pantallas, menús y HUD (HTML)
│   ├── css/               Estilos
│   ├── js/                Código (módulos ES)
│   └── assets/            Modelos, audio, imágenes, guion
├── dev-website/           Web y lanzador del equipo
├── database/
│   ├── Fafi_1_6.sql       Esquema de la base de datos
│   └── php-backend/       API, WebSocket, configuración, Docker y seeds
├── docs/                  Manuales, informe, Gantt, presentación, tráiler
├── herramientas/          Scripts de apoyo (optimizar modelos, voz...)
├── docker-compose.yml     Stack de producción
├── deploy.sh              Script de despliegue
└── .env.docker            Plantilla de variables de entorno
```

Versionado en el nombre de los archivos

Los archivos de código, estilos, herramientas y documentos llevan la versión en el nombre: `nombre_2_0_0v.ext` (por ejemplo `game_2_0_0v.js`, `style_2_0_0v.css`). Al subir de versión se renombran y se actualizan las referencias.

No se versionan (su nombre lo exige una herramienta o el servidor):

- `index.html` (lo busca Nginx), `Dockerfile`, `docker-compose.yml`, `composer.json`, `.htaccess`, `.gitignore`, `.env.docker`, `README.md`, `LICENSE`.
- Todo `database/` (esquema, seeds, PHP y configuración de Nginx/PHP): el despliegue del servidor depende de esos nombres.
- Los **assets** (modelos, audio, imágenes): el código arma muchas rutas por partes (por ejemplo `c1_0.wav`).

Reglas de orden: sin archivos duplicados, nombres sin errores de tipeo y los sonidos propios (`assets/audio/csgo/`) fuera del repositorio.

---

3. Cómo ejecutarlo

Solo el juego (sin servidor)

Servir `game-client` con cualquier servidor estático. Ejemplo:

```
cd game-client
python -m http.server 5173
```

Abrir http://localhost:5173 y usar **JUGAR SIN CONEXIÓN**. Abrir `index.html` con doble clic no funciona: el navegador bloquea los `.glb` y los módulos ES por `file://`.

Con servidor (Docker)

```
cp .env.docker .env          # completar DB_ROOT_PASSWORD, DB_PASSWORD, JWT_SECRET y DOMAIN
docker compose up -d --build
```

Verificar http://localhost/health. Ver la sección 12.

---

4. Arquitectura del cliente

4.1 Módulos (`game-client/js`)

Módulo	Función
`config` / `auth` / `api`	Scripts clásicos (no módulos): URL del servidor, sesión con JWT y llamadas a la API (`window.FAFI_CONFIG` , `authService` , `apiService`).
`game`	Motor principal (~4200 líneas): escena, bucle, entrada, HUD, armas, mapas, tienda y menús. Punto de entrada: `<script type="module">` .
`defuse`	Modo Defuse 5v5: rondas, economía, bomba, bots, marcador. **No importa** `game` : recibe un objeto `A` con lo que necesita.
`characters`	Personajes T/CT, animación por código (IK de piernas y brazos), armas de los bots.
`nav`	Malla de navegación para los bots (rayos contra colisiones + A* en grilla multinivel).
`zonas`	Zonas de spawn y sitios A/B de cada mapa.
`grenades`	Granadas: física con rebotes y efectos.
`skins` / `skinshader`	Catálogo de skins y shader de patrones.
`cajas`	Cajas: visor 3D y secuencia de apertura.
`menu2`	Inventario 3D y tienda de cajas.
`historia` / `ww2` / `modelos` / `storymap` / `mapa`	Modo historia: cinemáticas, modelos WW2, mapa de Europa.
`sfx`	Sonido (Web Audio, posicional).
`quality`	Calidad gráfica (mínima/baja/media/alta).
`loaders`	Cargador GLTF común (convierte `KHR_materials_pbrSpecularGlossiness`).
`chickens`	Gallinas decorativas con animación.

Todos los imports usan nombres versionados (`import ... from './defuse_2_0_0v.js'`). `index.html` lista los CSS y los tres scripts clásicos; `game` carga el resto.

4.2 Flujo de arranque

1. `index.html` muestra la pantalla de login. `auth` restaura la sesión de `localStorage` .
2. `game` inicia Three.js, carga el pack de armas y arma el menú (`fafi16-*`).

3. Al **entrar** (con servidor o sin conexión) se llama a `fafiApiLoadProfile()` si hay sesión real, y el menú se muestra.
4. Cada botón de modo (`#fafiPlayAim` , `#fafiPlayDefuse` , `#fafiPlayDust2` , `#fafiOpenCampaign`) inicia su flujo.

4.3 Bucle de juego

`requestAnimationFrame` llama al ciclo principal: **leer entrada** → **actualizar** (movimiento, colisiones, armas, bots, granadas) → **dibujar**. El arma y el cuchillo se dibujan en un **segundo pase** (capa 1) con una cámara de FOV fijo (84°), para que no se deformen con el FOV del mundo ni atraviesen paredes.

4.4 Calidad gráfica

`quality` aplica resolución de render, tipo y tamaño de sombras (el sol sigue al jugador), niebla, *tone mapping* y resolución de texturas de los mapas pesados. El nivel inicial se elige según memoria y núcleos (≤ 4 GB o ≤ 2 núcleos → mínima) y se guarda en `localStorage` (`fafi_quality_level`).

---

5. Modos de juego

5.1 Campo de tiro (AIM)

Sala simple (`aimMapGroup`) con blancos móviles, 30 s de tiempo. Al terminar se otorgan `floor(puntaje / 10)` monedas (`fafiDbAwardCoins` si hay servidor).

5.2 Defuse 5v5 (`defuse`)

- `createDefuse(A)` devuelve `{ D, startMatch, endMatch, update, bulletHit, meleeHit, areaDamage, ... }` . El motor le pasa en `A` la cámara, la escena, las colisiones, el mapa actual, callbacks de HUD y el callback `onMatchEnd` .
- Parámetros en `CFG` : congelado 10 s, ronda 105 s, bomba 40 s, 4 rondas para ganar.
- Cada unidad (`makeUnit`) tiene `hp` , `armor` , `money` , `kills` , `deaths` , `dmg` , `hs` .
- **Bots**: oyen disparos, comunican posición, hacen *strafe*, persiguen el último punto visto, rescatan o defienden la bomba y compran según su dinero. Se mueven con la malla de `nav` .
- **Daño y hitbox**: la bala es un segmento contra cápsulas del cuerpo (`segSeg` , `hitZone` , `bulletHit`); cabeza, torso y piernas tienen multiplicadores distintos; chaleco y casco absorben parte del daño.
- **Economía**: `CFG` define recompensas por ganar, perder (con racha) y por baja.
- Al terminar una partida se llama `A.onMatchEnd(resumen)` , que el motor reenvía a `fafiDbSaveMatch` (ver sección 11).

5.3 Mapas (`MAPS` en `game`)

```
MAPS = { dust2: { file, logo, baked, zones, nav, ... }, italy: {...}, office: {...} }
```

- Italy y Office usan versiones reducidas de `assets/Mapas para tt vs ct/optimizados` según la calidad.
- `zonas` define para cada mapa `t`, `ct` (spawn y zona de compra) y `A`, `B` (sitios) en coordenadas del mundo `(x, z)` con radio `r`.
- Si una zona es `null`, se calcula sola. `nav` ajusta `cell` y `headroom` de la malla de navegación.
- Italy y Office se ven de noche: `applyAtmosphere` / `tintMap` oscurecen los colores horneados, y hay cielo estrellado y niebla.

5.4 Mapa libre

Mismo motor que Defuse pero sin reglas: dinero infinito en la tienda (`B`), todas las armas habilitadas (`enableExtraWeapons(true)`).

---

6. Armas, personajes y skins

6.1 Armas

- **Pack por defecto**: un único GLB (`assets/armas por defecto/low_poly_weapon_free_pack.glb`) sin colores ni texturas. `characters` pinta cada triángulo por **zona** (madera, metal, polímero...) y `loaders` convierte sus materiales.
- **Armas extra**: `BOT_WEAPONS` describe cada GLB propio (`file`, `axis`, `length`). `bakeExtra` fusiona el modelo en **una sola malla** con color y zona por triángulo; `makeExtraWeapon` la coloca. Estas armas están **bloqueadas** en Defuse y habilitadas en exploración.
- **Viewmodel**: poses por arma en `buildPackWeapons` / `loadKnifeAnim`; el cuchillo trae sus propios brazos y animaciones.
- **Retroceso**: patrones por arma (`AK_PATTERN`, `M4_PATTERN`); la mira se abre según movimiento y disparo.
- **AWP**: zoom doble, cerrojo con tiempo de espera y mirilla.
- **Ranuras**: 1 primaria · 2 secundaria · 3 cuchillo · 4–7 granadas.

6.2 Personajes

T = Phoenix y Leet; CT = FBI y ST6. Los GLB no traen animaciones: `characters` las genera por código con **IK de dos huesos** en piernas y brazos (`animateCharacter`, `startThrow`, `startReload`, `hurtChar` ...).

6.3 Skins (`skins` , `skinshader`)

Una skin es una **receta**: `S(id, weapon, name, rarity, zones, pattern)`, con `P(type, colores, opciones)`.

```
S('ak_barroco', 'ak', 'AK-47 | Oro Barroco', 'legendaria', { wood: 0xc88f12, ... }, P('scales', [c1, c2, c3], { emit: 0.1 }))
```

- `zones` recolorea las zonas del arma; `pattern` aplica un **patrón procedural en el shader** (no usa UV): `camo`, `stripes`, `hex`, `flame`, `scales`, `fade`, `carbon`, `digital`, `tiger`, `tech`, `splatter`, `galaxy`, `waves`, `plates`.
- `WEAPON_ORDER` lista las armas habilitadas; `SKINS_DB` son las skins de esas armas (37) y `SKINS_LOCKED` las de armas bloqueadas (45 preparadas).
- Rarezas: `comun`, `epica`, `legendaria` (en `RARITY`).

---

7. Sistema de cajas

Las cajas viven en `cajas_2_0_0v.js` (visor y animación), `menu2_2_0_0v.js` (tienda) y `game_2_0_0v.js` (`fafiBuyBox`, lógica de compra y sorteo). El servidor tiene su propia versión del sorteo en `inventory.php`.

7.1 Datos

```
CRATES = {
  basic:     { name: 'CAJA BÁSICA',      price: 200,  odds: { comun: 85, epica: 15, legendaria: 0
}  },
  advanced:  { name: 'CAJA AVANZADA',    price: 500,  odds: { comun: 30, epica: 50, legendaria:
20 } },
  legendary: { name: 'CAJA CLASIFICADA', price: 1000, odds: { comun: 0,  epica: 50, legendaria:
50 } }
}
```

El modelo 3D es `assets/Cajas/optimizados/caja_1k.glb`, con su animación original.

7.2 Flujo de compra y sorteo (`fafiBuyBox`)

1. El jugador pulsa **ABRIR CAJA** en la tienda. Si no tiene monedas suficientes, se avisa y no pasa nada.
2. **Sin servidor (modo local):**
3. Se descuentan las monedas y se tira un número aleatorio para elegir la **rareza** según las probabilidades de la caja.
4. Se elige al azar una skin de esa rareza entre las 37 de `SKINS_DB`.
5. Si ya la tenía (**duplicada**), se devuelve un reembolso (50 común, 150 épica, 400 legendaria); si no, se agrega a `userInventory`.
6. `fafiSaveGameData()` guarda todo en `localStorage`.
7. **Con servidor:** `fafiDbBuyBox(type)` busca la caja en `GET /api/inventory/store` por nombre (*Caja Básica/Avanzada/Clasificada*) y llama a `POST /api/inventory/store/purchase`. **El servidor** (`handlePurchase` → `openLootBox`) descuenta las monedas, sortea la rareza y la skin, la guarda en `inventario_jugador` o reembolsa si estaba repetida, y responde `{ reward, coins }`. El cliente traduce el premio a una skin con `fafiDbSkinId` y actualiza monedas e inventario.
8. En ambos casos se llama a `openCrate({ skin, boxType, refund })` para mostrar la animación.

7.3 Animación de apertura (`openCrate`)

Se crea una capa HTML de pantalla completa (`position: fixed`) con su propio renderer Three.js. La secuencia está cronometrada con una tabla de tiempos `T` y se divide en fases:

1. **Entrada**: barras de cine, título de la caja y caída de la caja con golpe.
2. **Carga**: la caja vibra, se filtra luz por la tapa y las partículas se arremolinan hacia ella (más intensidad según la rareza: `tier` 1, 2 o 3).
3. **Apertura**: la tapa salta, con destello blanco y anillos de onda en el suelo.
4. **Premio**: el arma, construida con `buildSkinModel` , sale flotando; se muestra el nombre con el color de su rareza (`RARITY`) y, si era repetida, el reembolso.
5. **Cierre**: botón *ACEPTAR* o *SALTAR*; el renderer y la capa se destruyen.

Detalles técnicos útiles:

- La tapa y el asa **no tienen pieza propia en el GLB**: `buildCrate` detecta cuáles se mueven comparando posiciones de cada malla en dos instantes de la animación original y las cuelga de una **bisagra** propia en el borde trasero. Así `root.userData.open(angulo)` abre y cierra la caja por código.
- La caja se escala a 2,6 m de ancho y siempre se muestra de frente (con la etiqueta hacia la cámara).
- Los efectos (rayos, anillos, partículas, resplandor) son sprites con texturas generadas en un `canvas` (`glowTexture` , `beamTexture`). La cámara se acerca durante la carga y retrocede para el arma, con temblor.
- Los sonidos de la secuencia salen de `SFX` .

7.4 Tienda y visor

`menu2` arma la tienda con `crateThumb` (miniatura renderizada una vez y guardada en caché) y `createCrateViewer` (visor giratorio con un `setTier` que cambia el color del brillo). Debajo se listan las skins posibles con su porcentaje.

---

8. Modo historia y escena WW2

8.1 Visión general

El modo historia es un **mapa de Europa** (`storymap` , `mapa`) con 7 hechos. El primero, «El desembarco de Normandía», es una **cinemática 3D narrada** (`historia`). Los textos están en `assets/historia/guion.json` y la locución en `assets/historia/audio/cN_M.wav` (capítulo `N` , plano `M`).

Flujo: el jugador elige un hecho → `playChapter(id)` → al terminar, `onEnd(done)` marca el hecho como completado y desbloquea el siguiente.

8.2 Guion y duración

Cada capítulo del guion tiene `beats` (los textos de cada plano). **La duración de cada plano la marca el audio**: `audioDur()` lee la duración de cada `.wav` y `playChapter` calcula `bd = max(5.5, duración + 1.5)` segundos por plano. Así, cambiar la voz no obliga a tocar la animación.

8.3 Estructura de un capítulo (`CH[n]`)

Cada capítulo es una función `async` que recibe `X = { S (escenario 3D), A (audio), ov (capa HTML) }`, **construye los objetos una vez** y devuelve una lista de **planos**:

```
{ segs: [ { to: 0.5, kind: '3d', caption: '…', enter() {…}, update(u, dt) {…} },
          { to: 1.0, kind: 'photo', photo: 'landing', kb: [...] } ] }
```

- `kind` puede ser `'map'` (mapa 2D animado), `'photo'` (foto de época con movimiento de cámara *Ken Burns*), `'card'` (tarjeta de texto) o `'3d'` (escena 3D).
- `to` es la fracción del plano en que termina el segmento; `update(u, dt)` recibe el progreso `u` de 0 a 1.

8.4 El bucle del reproductor (`playChapter`)

1. **Carga**: duraciones de audio, fotos y modelos, con barra de progreso.
2. Se crea el escenario 3D (`createStage` : cielo, mar con olas animadas, luces y efectos) y el audio (música procedural + muestras).
3. **Cada cuadro**: calcula el tiempo transcurrido; elige el plano y el segmento actual; al cambiar de plano reproduce su `.wav` y muestra el subtítulo; ejecuta `update` ; dibuja la capa correspondiente (mapa, foto, tarjeta o 3D).
4. Al terminar, muestra *HECHO DESBLOQUEADO*, limpia todo (`cleanup`) y llama a `onEnd` . `Esc` o *Saltar* terminan antes.

8.5 Los modelos WW2 (`ww2`)

`FILES` define los GLB de `assets/WW2` y `PHOTOS` las tres fotos de contexto. Funciones principales:

Función	Qué hace
`loadGLB(clave)`	Carga y **cachea** cada modelo una sola vez.
`tame(root)`	Quita el exceso de metal: sin mapa de entorno los materiales metálicos salen negros, así que se limita `metalness`, se sube `roughness` y se anulan los mapas ORM.
`bakeStatic(root, animados)`	**Funde todas las mallas estáticas por material**: un modelo con cientos de piezas pasa a pocas llamadas de dibujo. Los nodos indicados en `animados` se conservan para poder moverlos.
`makeBoat()`	Lancha **Higgins** (11,3 m, proa hacia −Z). Conserva `door_102` (rampa) y `spin_196` (hélice). `userData.ramp(k)` baja la rampa interpolando entre la pose cerrada y la abierta.
`makeBismarck()`	Acorazado Bismarck (250 m).
`makeBomber()`	Bombardero con mezclador de animación propio (versión 1k en `optimizados`).
`makeGuard()`	Soldado alemán con animación (clonado con `SkeletonUtils`).
`makeMP40()`	MP-40 en primera persona con animación de recarga.
`makeDeco(clave, tamaño)`	Erizos checos, sacos de arena y cajas de munición.

Los modelos pesados se usan en versión reducida (`assets/WW2/optimizados`, generada con `optimizar-glb_2_0_0v.ps1`).

8.6 Cómo se arma la escena del desembarco (`CH[1]`)

- **Terreno**: `makeBeach` genera la playa y los acantilados con una **malla de alturas**; `terrH(x, z)` devuelve la altura para apoyar cada objeto.
- **Defensas**: 30 erizos checos, 5 búnkeres procedurales (`bunker()`), sacos de arena y 2 soldados alemanes por búnker.
- **Flota**: 2 filas de 13 lanchas Higgins, cada una con `x0`, `z0`, `stopZ`, velocidad y fase propias; `moveBoats(dt)` las hace avanzar y `S.ride()` las mece con las olas. Hay además una lancha "héroe" desde la que se ve la rampa caer.
- **Aviones**: 6 bombarderos que cruzan por encima.
- **Efectos**: `shelling()` lanza explosiones al azar, `defenderFire()` dispara trazadoras desde los búnkeres, `seaSmoke()` levanta humo.
- **Cámara**: `lookCam(cam, pos, objetivo)` coloca la cámara por plano; cada plano la interpola con `lerp` y `e01` según el progreso.
- **Planos del capítulo**: la flota cruza el Canal, los aviones abren el camino, las lanchas navegan, los defensores esperan, el mar se llena de explosiones, cae la rampa, los soldados cruzan Omaha y cae la noche con la playa tomada.

8.7 Audio de la cinemática

`createAudio` genera una **música procedural** en Re menor (cuerdas, timbales, caja de marcha, metales y arpegio) cuya intensidad sigue la función `MOOD[id]` según el avance del capítulo. `withSamples` suma muestras reales (mar, viento, motores, explosiones). La locución pasa por `mastered()`, una cadena de estudio (filtro grave, cuerpo cálido, presencia, compresión y un poco de sala) para que suene a narrador de documental.

8.8 Agregar un capítulo

1. Agregar su entrada en `guion.json` (`id`, `title`, `date`, `place`, `map`, `summary` y `beats`).
2. Generar o grabar sus audios `cN_M.wav` (ver Herramientas).
3. Escribir su constructor `CH[n]` en `historia_2_0_0v.js` (planos de mapa, foto o 3D).
4. Si usa 3D, agregarlo a `NEEDS_3D`.

---

9. Sonido

`sfx` usa **un solo** `AudioContext`. `DEF` mapea cada sonido a archivos y parámetros:

```
ak: { clips: [[archivo, inicio, duración], ...], v: 2.0, r: [0.96, 1.04], bus: 'wep' }
```

- **Buses** (`wep`, `foot`, `ui`, `mus`) con compresor y reverb sintética.
- `r` es el rango de velocidad (variación de tono) y `v` el volumen.
- **Posicional**: `SFX.at(nombre, vector3)` usa un `PannerNode`; `SFX.listener(camera)` lo orienta cada cuadro.
- Los archivos con extensión se cargan de `assets/audio/<archivo>`; el resto de `assets/audio/sfx/<nombre>.mp3`.
- **Paquete propio opcional**: si existe `assets/audio/csgo/manifest.json` (o recortes guardados desde `sonidos-csgo_2_0_0v.html`), esos sonidos reemplazan a los del juego. Esa carpeta está en `.gitignore` y no se sube.

---

10. Backend y base de datos

10.1 API REST (`database/php-backend/api`)

`index.php` enruta `/api/<recurso>/<acción>`. Las rutas protegidas requieren `Authorization: Bearer <JWT>`.

Recurso	Endpoints
`auth`	`POST /auth/register`, `POST /auth/login`, `GET /auth/me`, `PUT /auth/password`
`inventory`	`GET /inventory`, `GET /inventory/equipped`, `POST /inventory/equip`, `GET /inventory/store`, `GET /inventory/coins`, `POST /inventory/store/purchase`, `GET /inventory/history/purchases`
`social`	`GET/POST /social/friends`, `PUT /social/friends/{id}/accept`, `PUT /social/friends/{id}/block`, `DELETE /social/friends/{id}`, `GET /social/achievements`, `GET /social/search`
`game`	`GET /game/campaign`, `POST /game/campaign/complete`, `GET /game/stats`, `POST /game/stats/update`, `GET /game/leaderboard`, `GET/POST /game/matches`, `GET/PUT /game/config`, `POST /game/coins/earn`

El cliente las consume con `ApiService` (`api_2_0_0v.js`), que usa `authService.request`, el cual agrega el token y renueva la sesión.

10.2 Base de datos (`database/Fafi_1_6.sql`)

Tablas: `usuarios`, `configuraciones_jugador`, `progreso_campana`, `articulos_tienda`, `skins_armas`, `skins_soldados`, `inventario_jugador`, `historial_compras`, `estadisticas_armas`, `estadisticas_jugador`, `monedas_jugador`, `historial_partidas`, `partidas_jugadores`, `lista_amigos`, `logros`, `logros_desbloqueados`, `sesiones_juego`.

- `articulos_tienda`: cajas y skins con `precio` y `activo`. **No tiene clave única por nombre.**
- `skins_armas`: `articulo_id`, `arma_base` (`ak47`, `m4`, `sniper`, `deagle`, `knife`), `rareza` (`comun` / `epica` / `legendaria`) y `ruta_modelo` (`skin:<id>` para las skins actuales).
- El equipamiento se guarda en `configuraciones_jugador.preferencias_ui` (JSON `equipped_skins`).

10.3 Seeds

- `seed_mysql.sql`: datos iniciales (se carga solo en la primera creación de la base).
- `seed_skins_2_0_0v.sql`: catálogo actual de 37 skins, **idempotente** (usa `NOT EXISTS`). Se ejecuta a mano en servidores con la base ya creada.

---

11. Sincronización del juego con la base de datos

Todo está en `game_2_0_0v.js` (bloque «INTEGRACIÓN BASE DE DATOS»). `fafiDbOnline()` es verdadero solo con sesión real (no en modo sin conexión).

Qué	Cómo
Perfil al entrar	`fafiApiLoadProfile()` carga monedas, inventario, equipamiento, historial, amigos, logros, campaña y sensibilidad.
Skins	`fafiDbSkinId(fila)` traduce una fila de la base a un id del juego: primero por `ruta_modelo = 'skin:<id>'`, después por el mapa de nombres anteriores (`FAFI_DB_SKINNAME`) y por último por el nombre visible.
Armas	`FAFI_DB_WEAPON` traduce `arma_base` (`ak47`, `sniper`…) al id del juego.
Cajas	`fafiDbBuyBox` (ver sección 7): compra y sorteo en el servidor.
Equipar	`fafiDbPushEquipped` → `POST /inventory/equip`.
Monedas del campo de tiro	`fafiDbAwardCoins` → `POST /game/coins/earn`.
Campaña	`fafiDbCompleteCampaign` → `POST /game/campaign/complete`.
Partidas Defuse	`onMatchEnd` → `fafiDbSaveMatch` → `POST /game/matches` (mapa, equipo, ganador, bajas, muertes, daño, % de cabezas); el servidor actualiza el ranking.
Sensibilidad	`fafiDbSaveConfig` → `PUT /game/config`.

Sin servidor, el progreso se guarda en `localStorage` con la clave `fafi_save_<usuario>` (`fafiSaveGameData` / `fafiLoadGameData`).

Para agregar una skin nueva a la base de datos: crear la skin en `skins`, agregar sus filas a `seed_mysql.sql` y a `seed_skins_2_0_0v.sql` (artículo + `skins_armas` con `ruta_modelo = 'skin:<id>'`) y ejecutar el seed en el servidor.

---

12. Docker y despliegue

`docker-compose.yml` define tres servicios:

Servicio	Imagen	Función
`app`	`database/php-backend/Dockerfile`	Nginx + PHP-FPM + WebSocket (supervisord). Monta `game-client` y `dev-website` en solo lectura.
`db`	`mysql:8.0`	Base de datos; importa `Fafi_1_6.sql` y `seed_mysql.sql` al crearse.
`redis`	`redis:7-alpine`	Caché y sesiones (solo red interna).

Nginx sirve: `/` (el juego), `/dev-website/`, `/api` (PHP-FPM) y `/game` (WebSocket). Solo el puerto **80** queda expuesto.

Variables (`.env` , a partir de `.env.docker`): `DB_ROOT_PASSWORD` , `DB_PASSWORD` , `JWT_SECRET` (mínimo 32 caracteres) y `DOMAIN` .

```
cp .env.docker .env
docker compose up -d --build
curl http://localhost/health
```

`deploy.sh` (como root) hace una copia de seguridad de la base, actualiza el código, instala dependencias y levanta el stack.

Para actualizar el catálogo de skins en una base existente:

```
docker compose exec -T db sh -c 'mysql -u root -p"$MYSQL_ROOT_PASSWORD" fafi_1_6' < database/php-backend/database/seed_skins_2_0_0v.sql
```

---

13. Guías: cómo agregar contenido

Una skin nueva

1. En `skins_2_0_0v.js` , agregar `S('id', 'arma', 'Nombre', 'rareza', zonas, P('patron', [colores], opciones))` .
2. Para que funcione online, agregarla al seed (sección 11).

Un arma nueva

1. Dejar el GLB en `assets/` .
2. Registrarla en `BOT_WEAPONS` (`file` , `axis` , `length` , etiqueta) y en `skins` (`WEAPON_LABEL`).
3. Para habilitarla, agregarla a `WEAPON_ORDER` en `skins` (así pasa de `SKINS_LOCKED` a `SKINS_DB`) y a las listas de armas del motor.

Un mapa nuevo

1. Dejar el GLB (y su versión reducida en `optimizados` , con `optimizar-mapas_2_0_0v.bat`).
2. Agregar la entrada en `MAPS` y sus zonas en `zonas_2_0_0v.js` .
3. Ajustar `nav` (`cell` , `headroom`) si el mapa es cerrado.

Un capítulo de historia

Ver 8.8.

Una caja nueva

Agregar la entrada en `CRATES` (precio, color, probabilidades), un artículo `Caja …` en `articulos_tienda` y las probabilidades en `openLootBox` (`inventory.php`). El nombre debe coincidir con `FAFI_DB_BOX` .

---

14. Herramientas

En `herramientas/` :

Archivo	Uso
`optimizar-glb_2_0_0v.ps1`	Reduce las texturas de un GLB (versiones `optimizados`).
`optimizar-mapas_2_0_0v.bat`	Regenera los mapas reducidos de Italy y Office.
`generar-voz-piper_2_0_0v.ps1`	Genera la locución con Piper (voz neuronal sin conexión).
`generar-voz_2_0_0v.ps1`	Genera la locución con una voz de Windows.
`grabar-locucion_2_0_0v.html`	Graba una locución humana desde el navegador.

Además, `game-client/sonidos-csgo_2_0_0v.html` permite recortar y asignar sonidos propios de armas.

---

15. Pruebas y depuración

- **Consola del navegador**: el juego avisa con `console.warn` de fallos de carga y de sincronización (`Fafi DB: ...`).
- **Probar la integración con el servidor sin servidor**: sobrescribir `window.fetch` en la consola para simular `/api/...` (login, inventario, tienda y compra) y probar los flujos de `fafiApiLoadProfile` y `fafiBuyBox` .
- **Capturas automáticas**: con Chrome sin interfaz se puede abrir una página que controle el juego dentro de un `iframe` del mismo origen, simule clics y tome la captura (`--headless=new --screenshot`). Es la técnica usada para las imágenes de `docs/imagenes` .
- **WebGL sin GPU**: `--use-angle=swiftshader --enable-unsafe-swiftshader` .
- **Mouse bloqueado** (`PointerLockControls`): en un panel incrustado puede fallar; no afecta al resto del juego.

---

16. Límites conocidos y pendientes

- No hay cliente del **WebSocket**: el servidor ya expone `/game` , pero las partidas online todavía no están conectadas al juego.

- El modo historia tiene **un solo capítulo** terminado; los otros seis figuran en el mapa como próximos.
- Las 45 skins de armas bloqueadas no están en la base de datos hasta habilitar esas armas.
- Los mapas Italy y Office originales pesan más de 100 MB y no se suben a GitHub; el juego usa las versiones reducidas.
- Cambiar nombres de archivos de assets exige revisar las rutas armadas por partes en el código.
- Docker y el servidor PHP/MySQL no se probaron en la computadora de desarrollo; la integración se verificó con un servidor simulado en el navegador.